Natural Pion: Spectrum-Preserving Rotations along the Natural Gradient

Maxim Shikunov [1]

Abstract

Pion trains a language model by rotating every weight matrix on both sides, so its singular values stay fixed for the whole run, and it steers the rotation by the raw gradient, which treats every direction of rotation alike. We replace that direction with the natural gradient. On the rotation generators the K-FAC Fisher information has a closed form, and one generalized eigendecomposition per side inverts it exactly; we take the direction from the Fisher and the step length from Pion's own rule, and retract with a squared Cayley map, which keeps the spectrum at the larger angles these rotations reach. On LLaMA-60M trained on 9.6B tokens of C4, the resulting optimizer, **Natural Pion**, reaches Pion's losses in 1.9–2.1x fewer steps and lowers the full-length validation loss from 3.3706 to 3.3209, at 1.09x Pion's floating-point operations per step. Curvature preconditioning, long used to speed up optimizers that add to the weights, thus carries over to one that only rotates them.

1. Introduction

Most optimizers for neural networks update a weight matrix additively, $W \leftarrow W - \eta\,\Delta$, and differ in how they shape Δ from the gradient: Adam rescales it per coordinate, K-FAC and Shampoo precondition it with Kronecker factors, and Muon replaces it with its orthogonal polar factor. Pion (Shi et al., 2026) takes a different route: it updates W multiplicatively, by rotating it on both sides,

$$W \leftarrow R_{\text{out}}\, W\, R_{\text{in}}, \qquad R_{\text{out}} \in O(m),\ R_{\text{in}} \in O(n),$$

so the singular values of W are preserved throughout training. In particular, every weight keeps its initial spectral norm, with no weight decay or norm constraint needed. Pion reports that this makes training stable and competitive with Adam and Muon for LLM pretraining and finetuning.

[1]Independent researcher. Correspondence to: Maxim Shikunov <maks.shikunov2006@gmail.com>.

Preprint. November 11, 2026.

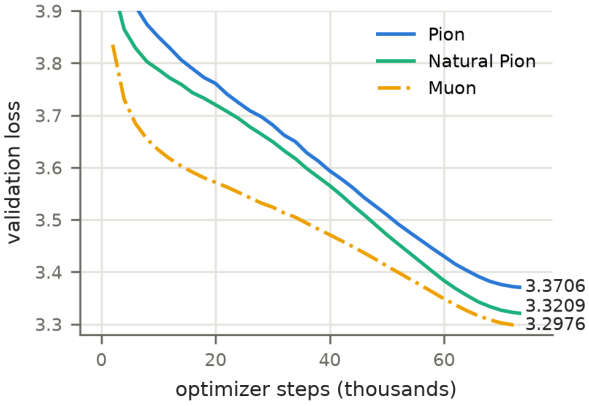


Figure 1. Validation loss against optimizer steps on LLaMA-60M / C4 at Pion's full budget (73 242 steps, 9.6B tokens), seed 0. Muon, which does not keep the spectrum, for reference.

Pion picks the rotation by steepest descent: it follows the gradient of the loss with respect to the two rotation generators. Steepest descent depends on the metric, and Pion's metric treats every direction of rotation alike. Yet the loss can be far more sensitive to some rotations of a matrix than to others. The classical remedy is the natural gradient (Amari, 1998), which measures steps by the Fisher information, the metric of the model's output distribution. In the K-FAC approximation, we derive the natural gradient for Pion's rotations and build on it an optimizer, **Natural Pion**.

Doing this directly runs into three obstacles. First, on the rotation generators the Fisher is not a Kronecker product, so K-FAC's inverse does not apply; solved as a dense system, it costs $O(n^6)$ per matrix. Second, the natural gradient gives a good direction but a bad scale: at a fixed step size, its rotation angles differ by a factor of $3.8 \cdot 10^7$ across the matrices. Third, the resulting rotations are about 4x larger than Pion's, and at such angles Pion's truncated exponential no longer preserves the spectrum.

Result. On LLaMA-60M trained on 9.6B tokens of C4, Pion's full budget, Natural Pion reaches Pion's losses in 1.90–2.12x fewer steps (Figure 3). At full length it lowers the validation loss from 3.3706 to 3.3209 (Figure 1),

at 1.09x Pion's floating-point operations per step. Muon, which does not preserve the spectrum, is still ahead, but Natural Pion closes about two thirds of Pion's gap to it (Section 4.2).

Contributions.

1. **The Fisher on rotation generators, in closed form.** Under the K-FAC approximation, the Fisher on each side's rotations is $F(X) = \frac{1}{2}(SXA + AXS)$ (Proposition 1). The reason is simple: one token's rotation gradient is a bivector. $F(X) = G$ is a generalized Lyapunov equation, which the classical simultaneous diagonalisation of the pencil (A, S) solves exactly with one generalized eigendecomposition per side (Proposition 2).
2. **Direction from the Fisher, length from Pion.** We use only the direction of $F^{-1}G$ and take the step length from Pion's RMS rule. With everything else fixed, this direction beats Pion's raw gradient by 0.132 in loss at 3 000 steps (Sections 3.3 and 4.4).
3. **A retraction that preserves the spectrum at large angles.** We apply Cayley steps by scaling and squaring, computed by Newton–Schulz with matrix products only. This keeps the spectrum to $3.7 \cdot 10^{-6}$ of each matrix's scale; Pion's truncated exponential moves it by $5.0 \cdot 10^{-3}$ (Section 3.4).
4. **A fast implementation.** Refreshing the curvature every 25 steps, collecting statistics only before a refresh, batching, and lower precision before the rotation make the step 2.5x faster than the exact form at the same loss (Algorithm 1, Section 4.5, Appendix C).

Code, run records and every check script cited here are at `https://github.com/Max0072/NaturalPion`; paths in typewriter type refer to that repository.

2. Background

2.1. Pion: training by orthogonal equivalence

Let $W \in \mathbb{R}^{m \times n}$ be a weight matrix with loss gradient $G = \partial L / \partial W$. Pion parameterises an update by two skew-symmetric generators, $\Omega_{\text{in}} \in \mathfrak{so}(n)$ and $\Omega_{\text{out}} \in \mathfrak{so}(m)$, and sets

$$W(\Omega) = e^{\Omega_{\text{out}}}\, W\, e^{\Omega_{\text{in}}}.$$

The derivative at $\Omega = 0$ is $\Omega_{\text{out}} W + W \Omega_{\text{in}}$. Write $\langle M, N \rangle = \operatorname{tr}(M^\top N)$ and $\operatorname{skew}(M) = \frac{1}{2}(M - M^\top)$. For skew Ω, $\langle M, \Omega \rangle = \langle \operatorname{skew}(M), \Omega \rangle$, so

$$dL = \langle G W^\top, \Omega_{\text{out}} \rangle + \langle W^\top G, \Omega_{\text{in}} \rangle$$
$$\Longrightarrow \quad G_{\text{in}} = \operatorname{skew}(W^\top G), \qquad G_{\text{out}} = \operatorname{skew}(G W^\top)$$

are the gradients of the loss with respect to the two generators (the Riemannian gradient on $O(m) \times O(n)$ at the identity).

Pion steps along $-G_{\text{in}}$ and $-G_{\text{out}}$ with three further ingredients:

- **Lie-algebra momentum.** An exponential moving average of G_{in} and G_{out}.
- **RMS scaling.** The step is scaled so that the change it makes to W, $W\Omega_{\text{in}} + \Omega_{\text{out}} W$, has a target RMS (0.2 times the learning rate).
- **A truncated exponential retraction** $e^{\Omega} \approx I + \Omega + \Omega^2/2$.

Pion applies both rotations at every step (the *bilateral* update); an alternating variant is slightly worse at 60M. Only the 2-D weights inside the transformer blocks are rotated; embeddings, the output head and normalization gains are trained by AdamW.

2.2. The natural gradient and K-FAC

For a model with predictive distribution $p_\phi(y \mid x)$, the Fisher information $F = \mathbb{E}[\nabla_\phi \log p \, \nabla_\phi \log p^\top]$ defines a metric on parameter space. The natural gradient $F^{-1}\nabla L$ is the steepest-descent direction in that metric (Amari, 1998):

$$-F^{-1}\nabla L \;\propto\; \arg\min_{\Delta} \langle \nabla L, \Delta \rangle \quad \text{s.t.} \quad \tfrac{1}{2}\Delta^\top F \Delta \le r.$$

In a language model the samples in this expectation are tokens. For each token t, a linear layer $y = Wx$ sees an input x_t and receives an output gradient $\delta_t = \partial \ell_t / \partial y_t$; its weight gradient is the average of the outer products $\delta_t x_t^\top$. K-FAC (Martens & Grosse, 2015) assumes x and δ are independent, which gives $F_W \approx A \otimes D$ with $A = \mathbb{E}_t[x_t x_t^\top]$ and $D = \mathbb{E}_t[\delta_t \delta_t^\top]$. As an operator on matrices, $F_W(Z) = DZA$.

Strictly, the Fisher averages over outputs y sampled from the model. Like most K-FAC implementations, we use the **empirical Fisher**: δ_t is the gradient of the training loss at the observed next token (Section 4.4 compares it with the true Fisher).

3. Method

3.1. The Fisher on rotation generators

By Section 2.1, token t's gradient with respect to $\Omega_{\text{in}} \in \mathfrak{so}(n)$ is $g_t = \operatorname{skew}(W^\top \delta_t \, x_t^\top)$. The (empirical) Fisher on $\mathfrak{so}(n)$ is the second moment of g_t, the operator $F_{\text{in}}(X) = \mathbb{E}_t[\langle g_t, X \rangle \, g_t]$. As a matrix it has about $1.7 \cdot 10^{10}$ entries for $n = 512$ and cannot even be stored. What makes it tractable is the form of g_t. With $u_t = W^\top \delta_t$, the output gradient pulled back to the input space,

$$g_t = \tfrac{1}{2}\big(u_t x_t^\top - x_t u_t^\top\big) = \tfrac{1}{2}\, u_t \wedge x_t, \quad \langle g_t, X \rangle = u_t^\top X x_t,$$

so g_t is a bivector, and its pairing with a skew X is a single bilinear form. Expanding the second moment and factoris-

ing it as K-FAC does leaves two $n \times n$ matrices. On the output side, δ_t and $v_t = W x_t$ play the roles of u_t and x_t.

Proposition 1 (Fisher on the generators). Under the K-FAC independence assumption, and treating the two sides separately,

$$F_{\text{in}}(X) = \tfrac{1}{2}\big(SXA + AXS\big), \qquad S = W^\top D W,$$

$$F_{\text{out}}(Y) = \tfrac{1}{2}\big(DYV + VYD\big), \qquad V = W A W^\top,$$

for $X \in \mathfrak{so}(n)$, $Y \in \mathfrak{so}(m)$. Both operators are self-adjoint and positive semi-definite on the skew matrices: $\langle Z, SXA + AXS\rangle$ is symmetric in X and Z, and $\langle X, F_{\text{in}}(X)\rangle = \mathbb{E}[(u^\top X x)^2] \geq 0$ before the factorisation and $\frac{1}{2}\langle X, SXA + AXS\rangle = \|S^{1/2} X A^{1/2}\|_F^2 \geq 0$ after it. The derivation is in Appendix A.

Relation to K-FAC. This operator is K-FAC itself, written on the rotation generators. A generator changes W by $J(\Omega) = W\Omega$, and the adjoint of this map is $J^*(M) = \operatorname{skew}(W^\top M)$. Pulling K-FAC's Fisher on W, $F_W(Z) = DZA$, back through J gives

$$\begin{aligned} J^* F_W J(X) &= \operatorname{skew}(W^\top D\, W X\, A) \\ &= \operatorname{skew}(SXA) = \tfrac{1}{2}\big(SXA + AXS\big), \end{aligned}$$

which is Proposition 1. The result is not a Kronecker product, so K-FAC's inverse $D^{-1} Z A^{-1}$ does not apply; Section 3.2 inverts it instead.

What is approximated. Three things: x and δ are treated as independent, as in K-FAC; the two sides and the layers are treated separately; and the empirical Fisher stands in for the true one (Section 2.2). The rest is exact algebra, which Appendix A also checks numerically; how far K-FAC is from the exact curvature along our step is measured in Section 3.3.

3.2. Exact inversion

F_{in} acts on a space of dimension $n(n-1)/2$. Solving $F_{\text{in}}(X) = G$ as a dense linear system costs $O(n^6)$, about 10^{15} operations for $n = 512$, for every matrix at every refresh. Nor can it be inverted factor by factor like $X \mapsto SXA$, whose inverse is $S^{-1} G A^{-1}$: the sum of the two orderings does not factor unless S and A commute. A congruence diagonalises it nonetheless, at $O(n^3)$: $SXA + AXS = 2G$ is a generalized Lyapunov equation, and simultaneous diagonalisation of the pencil (A, S) is the standard way to solve it (Golub & Van Loan, 2013; Simoncini, 2016). We state the solution for completeness.

Proposition 2 (Inversion). Let $A \succ 0$ and $S \succeq 0$, and let P solve the generalized eigenproblem $P^\top A P = I$, $P^\top S P = \Lambda = \operatorname{diag}(\lambda)$, with $\lambda_i + \lambda_j > 0$ for all $i \neq j$ (the diagonal of a skew matrix is zero). For skew G, the unique skew solution of $F_{\text{in}}(X) = G$ is

$$X = P\left[(P^\top G P) \oslash K\right] P^\top, \qquad K_{ij} = \tfrac{1}{2}(\lambda_i + \lambda_j),$$

where $\oslash$ is elementwise division.

Proof. Substitute $X = P\tilde{X}P^\top$: $P^\top(SXA + AXS)P = (P^\top S P)\,\tilde{X}\,(P^\top A P) + (P^\top A P)\,\tilde{X}\,(P^\top S P) = \Lambda\tilde{X} + \tilde{X}\Lambda$. So $F_{\text{in}}(X) = G$ becomes $\frac{1}{2}(\lambda_i + \lambda_j)\,\tilde{X}_{ij} = (P^\top G P)_{ij}$. Since $P^\top G P$ is skew, so are $\tilde{X}$ and X. P is invertible, so the solution is unique. □

The same construction applies to the output side, with A replaced by D and S by $V = WAW^\top$. A and D need not be positive definite; the floor below makes them so, as the proposition requires.

Computation. We compute $P = A^{-1/2} Q$, with Q the eigenvectors of $A^{-1/2} S A^{-1/2}$. The symmetric root, rather than a Cholesky factor, exposes the eigenvalues so they can be floored.

Regularisation. A single relative floor: every spectrum the solve uses (A, S, Λ on the input side; D, V, Λ on the output side) is raised to at least $\varepsilon\lambda_{\max}$, with $\varepsilon = 10^{-3}$. Unlike an additive shift, a floor leaves every eigenvalue above it unchanged.

Cost. One generalized eigendecomposition per side, $O(n^3)$, at every refresh (Algorithm 1 amortises it); applying F^{-1} between refreshes costs four matrix products per side.

3.3. Step length

The natural gradient gives a direction $X = F^{-1}G$, but its scale is a poor guide. At a fixed step size, the rotation angle it produces differs by a factor of $3.8 \cdot 10^7$ across the rotated matrices (90th over 10th percentile, Section 4.4). The scale is also where K-FAC is least reliable: along our step, its curvature is 0.029 of the exact per-token curvature on a trained model (`scripts/probes/paper_checks.py`). So we keep the direction and take the length from Pion: the coefficient c sets the RMS of the change the step makes to W to the scheduled θ_t,

$$c_t = \theta_t \, \frac{\sqrt{mn}}{\|W X_{\text{in}} + X_{\text{out}} W\|_F},$$

the same rule Pion applies to its raw generators. Since c is set per matrix, the Fisher's scale across matrices is discarded. What the Fisher still decides is the direction within each matrix and, since both sides share c, the balance between its two sides. To first order in the angle, each rotation plane then turns in proportion to the corresponding singular value of X. θ_t follows a cosine schedule from a peak θ_0. Section 4.4 compares this rule with fixing the rotation angle ($c = \theta/\sigma(X)$) and with measuring the length in the Fisher metric.

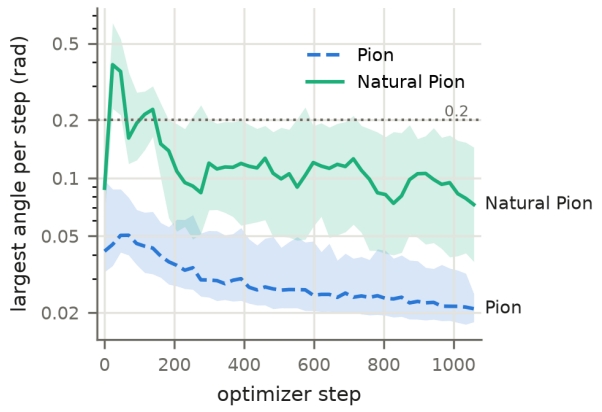


Figure 2. Largest rotation angle per rotated matrix and step: median over the matrices, band from the 10th to the 90th percentile, log scale. First 1 058 steps, where both were recorded and the learning rate is near its peak. Natural Pion at 3 000 steps, seed 0; Pion at lr 10^{-3} on a 3 840-step schedule (`scripts/probes/pion_angles.py`). Dotted: 0.2, above which the squared Cayley retraction splits the step (Section 3.4).

Proposition 3 (Angle of a Cayley step). Let X be skew with largest singular value σ and $c > 0$. The rotation $R = \left(I + \frac{c}{2}X\right)^{-1}\left(I - \frac{c}{2}X\right)$ turns its planes by the angles $2\arctan(c\,\sigma_j/2)$, where σ_j are the singular values of X. Its largest angle is $2\arctan(c\sigma/2) = c\sigma + O((c\sigma)^3)$.

Proof. A real skew matrix has eigenvalues $\pm i\sigma_j$, in planes that are mutually orthogonal. On such a plane R acts by $(1 - ic\sigma_j/2)/(1 + ic\sigma_j/2) = e^{-2i\arctan(c\sigma_j/2)}$. □

Under the RMS rule $c\sigma$ is not fixed: a plane that carries little of W barely moves it, so it may turn far. At the worst logged step, early in training, a tenth of the matrices turn their fastest plane by 0.64 rad or more, and the retraction has to handle this. Over the first thousand steps, where both are measured, the median angle is 4.0 times Pion's (range 2.7 to 8.6; Figure 2).

3.4. Retraction: the squared Cayley map

Proposition 4. For skew Ψ, the Cayley map $R = (I + \Psi)^{-1}(I - \Psi)$ is orthogonal for every scale of Ψ. Hence $R_{\text{out}} W R_{\text{in}}$ has exactly the singular values of W. The degree-2 truncated exponential $T = I + \Psi + \Psi^2/2$ satisfies $T^\top T = I + \Psi^4/4$.

Proof. The eigenvalues of a real skew Ψ are $\pm i\sigma_j$, so those of $I + \Psi$ are $1 \pm i\sigma_j \neq 0$ and $I + \Psi$ is invertible. $I + \Psi$ and $I - \Psi$ commute, and $(I \pm \Psi)^\top = I \mp \Psi$, so $R^\top R = (I+\Psi)(I-\Psi)^{-1}(I+\Psi)^{-1}(I-\Psi) = I$. For T, expand with $\Psi^\top = -\Psi$: the odd and quadratic terms cancel, leaving $\Psi^4/4$. □

What the retraction must do. It must stay orthogonal to round-off at every angle, on every matrix and every step. Pion's angles are smaller and more uniform: the median over the rotated matrices is 0.026 rad per step (p90/p10 2.0), the largest 0.097 (`scripts/probes/pion_angles.py`, first 1 058 steps). By Proposition 4, one truncated-exponential step at angle a changes a singular value by $\sqrt{1 + a^4/4} - 1 \approx a^4/8$. That is about $6 \cdot 10^{-8}$ per step at Pion's typical angle, but $4 \cdot 10^{-7}$, $9 \cdot 10^{-6}$ and $2 \cdot 10^{-2}$ at ours (3 000 steps; the run's median of the 50th and 90th percentiles over the matrices, and the largest 90th percentile).

Cayley by Newton–Schulz. The Cayley map needs a linear solve: $\text{Cay} = 2(I + \Psi)^{-1} - I$ with $\Psi = \frac{c}{2}X$. Newton–Schulz computes $(I + \Psi)^{-1}$ with matrix products only, $Z \leftarrow Z(2I - (I + \Psi)Z)$ from $Z_0 = I - \Psi$. After two iterations the error is $2(I + \Psi)^{-1}\Psi^8$: the largest entry of $R^\top R - I$ is $5 \cdot 10^{-9}$ at angle $a = c\|X\|_2 = 0.2$, 10^{-6} at 0.4, $3 \cdot 10^{-4}$ at 0.8 (fp64).

In fp32 this standard form is biased. Entries close to 1 lose their low bits, always in the same direction, so $\|W\|_F^2$, which a rotation should keep, grows by a median $6.8 \cdot 10^{-6}$ per training step (exact solve: $2.0 \cdot 10^{-7}$; `scripts/probes/ns_diag.py`). We therefore iterate on $\Phi = Z - I$, from $\Phi_0 = -\Psi$: residual $E = \Psi + \Phi + \Psi\Phi$, update $\Phi \leftarrow \Phi - E - \Phi E$, rotation $I + 2\Phi$. No entry is then close to 1, and the number of products is the same. Appendix D checks the bias over random rotations.

Scaling and squaring. Two iterations reach fp32 accuracy only up to an angle of about 0.2, and the RMS rule takes larger steps. Natural Pion therefore halves the step's largest angle (estimated from σ, Appendix C.5) q times until it is at most 0.2, runs the two iterations on $\Psi/2^q$, and squares the result q times. The squaring uses the same correction form, $H \leftarrow 2H + H^2$ for $R = I + H$, so again nothing is rounded near 1. Each squaring costs one product, less than the extra iterations or the exact solve it replaces (`scripts/probes/cayley_speed.py`).

The result, $R = \text{Cay}(cX/2^q)^{2^q}$, is no longer the Cayley transform of cX. Since Cayley is the $[1/1]$ Padé approximant of the exponential, it is the scaling-and-squaring method for the matrix exponential (Higham, 2005) with that approximant, an approximation of the geodesic step on $O(n)$. We call it the squared Cayley retraction. On the plane of singular value σ_j it turns by $2^{q+1}\arctan(c\sigma_j/2^{q+1})$; with $y = c\sigma_j/2^q \leq 0.2$, this matches the exponential's angle $c\sigma_j$ to a relative $y^2/12 \leq 1/300$ at any step length (Proposition 3 applies to each factor, not to R). With exact Cayley factors R is orthogonal, because a diagonal Padé approximant maps skew matrices to orthogonal ones; a truncated Taylor series, Pion's approximation of the same exponential, does not (Proposition 4). In practice each factor carries the error of its Newton–

Schulz inverse, and the squarings multiply it by up to 2^q. In training the spectrum moves by $3.7 \cdot 10^{-6}$ of its scale over 3 000 steps (Table 1). For most of the run the 90th percentile of the angle is below 0.2 (median 0.09 rad), so the squarings matter mainly early in training.

At 3 000 steps the validation losses of the four retractions lie within 0.010 of each other (one run each; the method's four seeds span 0.006); at 15 000 steps the truncated exponential is -0.0006 from the method. So the retraction decides the spectrum, not the loss: the squared Cayley retraction moves it by $3.7 \cdot 10^{-6}$, the others by $4.0 \cdot 10^{-4}$ to $5.2 \cdot 10^{-3}$.

Precision. The retraction must run in true fp32: over 200 exact Cayley steps at angle 10^{-2} on a 512×1376 matrix, TF32 moves a singular value by up to 0.37% (relative), fp32 by 0.012%.

3.5. The algorithm

Algorithm 1 is Natural Pion as run in every experiment. Like K-FAC with its inverse (Martens & Grosse, 2015, Sec. 13), it amortises the bases: they are recomputed every $T_{\text{fac}} = 25$ steps, and A and D are accumulated only in the 6 steps before each refresh, since nothing else reads them. F^{-1}, the step's scale and the retraction run as one batched call per matrix shape, and the largest angle σ comes from a warm-started power iteration. The *exact form*, our reference in Section 4.5 and Appendix C, recomputes everything at every step ($T_{\text{fac}} = 1$, A with EMA 0.95 and D with 0.5, both every step) and uses the exact Cayley solve $\text{Cay}(cX) = (I + \frac{c}{2}X)^{-1}(I - \frac{c}{2}X)$.

As in Pion, embeddings, the output head and normalization gains use AdamW. The rotated matrices get no weight decay: a rotation cannot change their norm.

Under the RMS rule, a constant factor in A or D (such as autograd's $1/N$) cancels in cX; the Fisher-norm and quadratic-model rules and the damping of Section 4.4 need D at its true scale (Appendix D).

4. Experiments

4.1. Setup

Model and data. A LLaMA-style model of 60M parameters (hidden 512, 8 layers, 8 heads, SwiGLU FFN, RoPE, RMSNorm, T5 vocabulary) on C4, sequence length 256, batch 512 sequences (131 072 tokens per step), for 73 242 steps = 9.6B tokens: Pion's 60M budget. The 56 matrices inside the blocks (43.5% of parameters) are rotated; the rest are trained by AdamW (lr $8 \cdot 10^{-3}$, betas (0.9, 0.95), weight decay 0.1), the same for every method. The metric is validation loss, token cross-entropy on held-out C4 in nats; each run uses one NVIDIA RTX PRO 6000 Blackwell. Appendix B gives the full setup (Table 10) and hyperparameters (Table 12).

Baseline. Pion's rotation update in its published settings: Lie momentum 0.9, RMS scale 0.2, truncated exponential, bilateral update, cosine schedule. The Pion paper chose lr 10^{-3} from $\{10^{-4}, 5 \cdot 10^{-4}, 10^{-3}, 5 \cdot 10^{-3}, 10^{-2}\}$. Since Natural Pion's scale is chosen per budget, so is Pion's lr: from 15 000 steps on we take the better of 10^{-3} and $2 \cdot 10^{-3}$ (seed 0) at every budget. Table 11 (Appendix B) shows a wider grid at 3 840 and 19 200 steps.

All comparisons use one code base and one data pipeline (Muon's run below uses a copy of it). In our setting Pion ends at a training loss of 3.3735 (mean over the last 2 000 steps; validation 3.3706). The Pion paper reads 3.3575 off its own training curve in Megatron-LM (Shoeybi et al., 2019). A single batch's training loss varies by 0.026 here, so the two numbers are not directly comparable, and we compare within our setting only.

Natural Pion. Algorithm 1, Lie momentum 0.9, floor $\varepsilon = 10^{-3}$. Every schedule of every method, θ and AdamW's lr alike, is a cosine down to 1% of its peak (for Pion at lr 10^{-3}, 10^{-5}). The peak RMS scale θ_0 is chosen per horizon on a grid of factors of two: $6.0 \cdot 10^{-4}$ at 3 000 steps, $3.0 \cdot 10^{-4}$ at 9 000, $3.0 \cdot 10^{-4}$ at 15 000 and $3.0 \cdot 10^{-4}$ at 30 000; the longer budgets keep the 30 000-step choice. These peaks are close to Pion's own target, $0.2 \cdot \text{lr} = 2 \cdot 10^{-4}$ to $4 \cdot 10^{-4}$: per step, Natural Pion changes W by about as much as Pion does, so its gain comes from the direction of the step, not from a larger one.

4.2. Main result

Figure 1 shows the full-length curves. At 3 000 steps the smallest per-seed gap is 0.153. Seed 0 was also used to choose Natural Pion's scale; over seeds 1–3 alone its mean is 3.6839 at 3 000 steps and 3.4482 at 15 000.

Muon, for reference. Muon (Jordan et al., 2024), which does not preserve the spectrum, in the Pion authors' configuration (lr 10^{-3}, same AdamW and schedule; micro-batch 512) reaches 3.2976 at full length: 0.073 below Pion and 0.023 below Natural Pion. The Pion paper also reports Muon ahead of Pion (its Table 1, 1.3B). Natural Pion thus closes 68% of Pion's gap to Muon, and 66% against Pion at its best full-length AdamW learning rate (Table 3). Muon is not re-tuned here, so a tuned Muon would lower that share; how much of the remaining gap is the price of preserving the spectrum, we do not know.

AdamW at full length. AdamW's lr (embeddings, head, norms) was chosen at 3 000 and 15 000 steps. At full length, the best of the three values in Table 3 is $4 \cdot 10^{-3}$ for Pion and $8 \cdot 10^{-3}$ for Natural Pion, 0.004 and 0.000 better than $8 \cdot 10^{-3}$. The gap stays between 0.042 and 0.050 (seed 0;

Table 1. Retractions: Natural Pion (Algorithm 1) with each, seed 0. First column: change of $\|W\|_F^2$ over 200 steps of a bare 1376×1376 rotation at angle 0.05 (`scripts/probes/ns_rounding.py`). Drift: $\max_i |s_i - s_i^0| / \max_i s_i^0$ after 3 000 training steps, median / worst over the rotated matrices (`scripts/probes/retraction_drift.py`). –: not run.

retraction	$\Delta\|W\|_F^2$, bare matrix	val, 3 000 steps	val, 15 000 steps	drift	s / step
Cayley, exact solve	$+3.4 \cdot 10^{-5}$	3.6826	–	$4.0 \cdot 10^{-4}$ / $4.4 \cdot 10^{-4}$	0.533
Cayley, Newton–Schulz (standard form, iterations growing with the angle, no squaring)	$+2.8 \cdot 10^{-4}$	3.6775	–	$5.2 \cdot 10^{-3}$ / $1.9 \cdot 10^{-2}$	0.495
squared Cayley (Natural Pion)	$+1.6 \cdot 10^{-7}$	3.6878	3.4465	$3.7 \cdot 10^{-6}$ / $4.8 \cdot 10^{-6}$	0.498
truncated exponential, degree 2 (Pion's)	–	3.6830	3.4459	$5.0 \cdot 10^{-3}$ / $2.2 \cdot 10^{-2}$	0.483

Algorithm 1 Natural Pion, for one rotated matrix $W \in \mathbb{R}^{m \times n}$ (all of them processed alike, batched by shape).

```
state:  A, D: EMAs (0.5, fp32) of x x^T and d d^T;  M_in, M_out: momentum (beta1 = 0.9), zero at start;
        bases (P, lambda) per side;  eps = 1e-3,  T_fac = 25
every step t:
    theta_t <- theta_min + (theta_0 - theta_min)(1 + cos(pi t / T)) / 2     cosine; theta_min = theta_0 / 100
    if t is one of the 6 steps before a refresh:                 one A per input tensor; bf16 inputs, fp32 sums
        forward:  A <- 0.5 A + 0.5 mean_tokens(x x^T)
        backward: D <- 0.5 D + 0.5 mean_tokens(d d^T)             d = per-token output gradient
    if t is a refresh step (every T_fac steps):                  the bases (Prop. 2); spectra floored at eps * max
        (P_in, lambda_in)   <- gen_eig(A, W^T D W)
        (P_out, lambda_out) <- gen_eig(D, W A W^T)
    G_in  <- skew(W^T G);   G_out <- skew(G W^T)
    M_in  <- beta1 M_in  + (1 - beta1) G_in;   M_out <- beta1 M_out + (1 - beta1) G_out
    X_in  <- P_in  [ (P_in^T  M_in  P_in)  / K_in  ] P_in^T     K_ij = (lambda_i + lambda_j)/2; products in bf16
    X_out <- P_out [ (P_out^T M_out P_out) / K_out ] P_out^T
    c     <- theta_t sqrt(m n) / ||W X_in + X_out W||_F          Pion's RMS rule
    sigma <- max(||X_in||_2, ||X_out||_2)                        warm power iteration (2 steps; 50 after refresh)
    q     <- smallest q >= 0 with c sigma / 2^q <= 0.2           one q per step: the largest c sigma of all matrices
    W     <- SqCay(c X_out) W SqCay(c X_in)          in fp32; SqCay(cX) = Cay(cX / 2^q)^(2^q),
                                                     each Cay by 2 Newton-Schulz steps in Z - I (Sec. 3.4)
```

Table 2. Equal number of steps (seed 0 unless marked). Pion at the better of lr 10^{-3} and $2 \cdot 10^{-3}$ at each budget (Section 4.1); at 3 000 steps 10^{-3}. Gap = val(Pion) - val(Natural Pion).

steps	Pion	Natural Pion	gap
3 000 (mean of 4 seeds)	3.8394	3.6848	0.155
15 000	3.5254	3.4465	0.079
73 242 (full length)	3.3706	3.3209	0.050

Table 3. Full length, by AdamW lr.

AdamW lr	Pion	Natural Pion	gap
10^{-3}	3.3786	3.3366	0.042
$4 \cdot 10^{-3}$	3.3667	3.3235	0.043
$8 \cdot 10^{-3}$ (the rest of the paper)	3.3706	3.3209	0.050

Pion at the better of lr 10^{-3} and $2 \cdot 10^{-3}$ for each AdamW lr).

4.3. Steps to reach Pion's loss

Every run here has its own full cosine schedule ending at its budget B, so no loss is read off the middle of a schedule. For each method we fit the final validation loss as $L(B) = a + bB^{-\gamma}$ (9 Pion budgets, 6 Natural Pion budgets, four seeds at 3 000 and 15 000 steps; worst residual 0.0021). The saving at Pion's loss L is $B_{\text{Pion}}/B_{\text{NP}}(L)$, with B_{NP} taken from the fit or by linear interpolation in $\log B$ between Natural Pion's budgets (`scripts/analysis/budget_fit.py`). Figure 3 shows both fits.

Table 4. Steps saved at Pion's loss (mean over seeds where there are four): fit / interpolation.

Pion steps	Pion val	saving
15 000	3.5254	1.90 / 1.84
19 200	3.4945	1.92 / 1.88
30 000	3.4460	1.97 / 1.96
45 000	3.4050	1.96 / 1.93
73 242	3.3706	2.12 / 2.16

Pion's targets use its better lr of 10^{-3} and $2 \cdot 10^{-3}$ at each budget (Section 4.1). The gap in loss at equal steps shrinks with the budget (0.155, 0.079 and 0.050 at 3 000, 15 000 and 73 242 steps, Table 2) because both curves flatten. Measured in steps, the advantage does not shrink: 1.90x at 15 000 steps, 2.12x at full length.

Equal wall-clock time. A Natural Pion step takes 1.08x

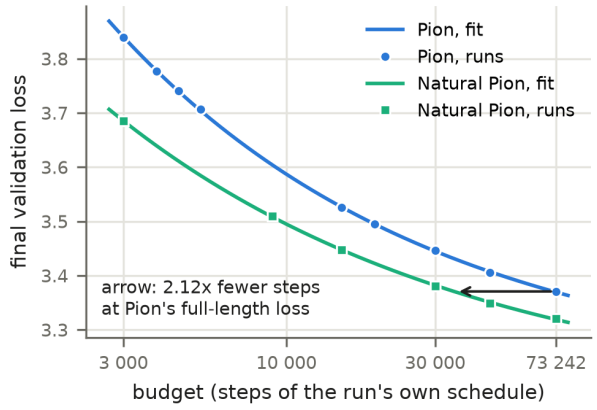


Figure 3. Final validation loss of every full-schedule run against its budget, with the fit $L(B) = a + bB^{-\gamma}$ of each method; at 3 000 and 15 000 steps the mean of four seeds. The arrow is the saving at Pion's full-length loss.

Table 5. The direction. Pion at 3 000 steps, seed 0: 3.8412.

	no momentum	Lie momentum 0.9
$X = F^{-1}G$	3.7590	3.6878
$X = G$	4.2223	3.8199
gain from F^{-1}	0.463	0.132

Pion's (Section 4.5), so in the time of Pion's B steps Natural Pion takes $B/1.08$. Read off the budget curve above (estimates, not separate runs), the gap to Pion at Pion's full-length wall-clock time is 0.047 (fit; 0.045 by interpolation), 0.065 at the time of 19 200 steps, and 0.124 at the time of 3 840 steps.

4.4. Ablations

Each ablation changes one thing in Natural Pion (Algorithm 1) at 3 000 steps, seed 0. The reference is the method itself: RMS scale $6.0 \cdot 10^{-4}$, Lie momentum 0.9, floor 10^{-3} (3.6878). "Best" means the best over a grid of that row's step-size parameter.

The direction: natural gradient against the raw generator. Only $X = F^{-1}G$ against $X = G$ changes; the length rule, retraction and schedule are the same, each at the best scale of its grid.

The length. Only the length rule changes, each at the best θ of its grid. The last column is the spread of the realised rotation angle across the rotated matrices (90th over 10th percentile, median over the steps after 1 000). In the quadratic-model and Fisher-norm rules, F is the Fisher at the current W with the current statistics. With momentum, $X = F^{-1}M$, so with a fresh basis $\langle X, FX\rangle = \langle M, X\rangle$ and the quadratic-model ratio is $\langle G, X\rangle / \langle M, X\rangle$. It differs from a fixed c only where the current gradient disagrees with the momentum or the basis has gone stale.

Table 6. The length rule.

length rule	val	angle spread
RMS of the induced change, as in Pion (the method)	3.6878	$2.8 \cdot 10^{0}$
rotation angle: $c = \theta/\sigma(X)$, $X = F^{-1}M$	3.6892	$1.0 \cdot 10^{0}$
quadratic-model ratio, $c = \theta \min(1, \langle G, X\rangle / \langle X, FX\rangle)$	3.7260	$8.1 \cdot 10^{1}$
Fisher norm, $c = \theta / \sqrt{\langle X, FX\rangle}$	3.7343	$1.0 \cdot 10^{2}$
fixed, $c = \theta$	3.9946	$3.8 \cdot 10^{7}$
rotation angle before F^{-1}: $c = \theta/\sigma(M)$	4.5456	$1.0 \cdot 10^{9}$

Table 7. Regularisation of the Fisher.

floor ε (no damping)	10^{-6}	10^{-5}	10^{-4}	10^{-2}
val	3.7987	3.7465	3.6927	3.7176
damping μ (floor 10^{-6}; $F + \mu\lambda_{\max}(B)\, BXB$, B = A in, D out)	10^{-4}	10^{-3}	10^{-2}	10^{-1}
val	3.7068	3.6947	3.7124	3.7423

Over four seeds, each at the best θ of its grid, the RMS rule minus the angle rule is -0.001, $+0.000$, -0.003 and -0.001: the two are within noise of each other, and we keep Pion's rule.

Regularisation of the Fisher. The default is a relative floor $\varepsilon = 10^{-3}$ (3.6878).

The best alternative is $+0.005$ from the default.

The true Fisher. Sampling the targets from the model (D from sampled outputs, at the cost of one more forward and backward pass) gives 3.6815 at the best scale of its grid, against 3.6878 for the empirical Fisher.

4.5. Cost

The exact form (Section 3.5) takes 1.22 s per step in our setting, against Pion's 0.473 s. About half of it is the basis refresh, two generalized eigendecompositions per matrix at every step (Table 13). Algorithm 1 brings it to 0.498 s with five changes; Appendix C gives what each saves and what it costs in loss:

- **Amortised refresh** (C.2). The bases are recomputed every 25 steps, as K-FAC recomputes its inverse every 20 iterations.
- **Statistics only where they are read** (C.3). A and D are accumulated (EMA 0.5) only in the 6 steps before a refresh.

Table 8. From the exact form to Algorithm 1 (s / step: time between logged steps of each training run, evaluation excluded; val at 3 000 steps, seed 0; the last row groups the changes of Appendix C.4–C.6 and Section 3.4).

step	s / step	val
exact form	1.22	3.6860
refresh every 25 steps (C.2)	0.668	3.6903
+ statistics only before a refresh (C.3)	0.568	3.6867
+ batching, one A per input, bf16 statistics, lower precision before the rotation, squared Cayley: Algorithm 1	**0.498**	**3.6878**

- **Batching** (C.4, C.5). F^{-1}, the Cayley step and the RMS scale run as one call per matrix shape; the step's largest angle comes from a warm-started power iteration.
- **Cheaper statistics, lower precision before the rotation** (C.6). One A per input tensor, and bf16 inputs to the grams with fp32 accumulation. Products that only set the direction run in bf16 and TF32; the rotation itself runs in fp32.
- **The squared Cayley retraction** (Section 3.4): matrix products instead of the exact Cayley solve.

Over four seeds, the exact form averages 3.6840 and Algorithm 1 3.6848; at 15 000 steps (seed 0, both at the method's 15 000-step scale), 3.4458 against 3.4465. One change alters the step itself, not only its cost: the exact Cayley step turns a plane by $2\arctan(c\sigma_j/2)$, the squared Cayley step by about $c\sigma_j$ (Section 3.4). A step takes 1.08x Pion's time on our GPU.[1] Its peak GPU memory is 43.02 GB against Pion's 42.26 GB ($+0.76$ GB for A, D, the bases and the momentum).

Floating-point operations. Seconds depend on the GPU and the implementation; operation counts do not. Table 9 counts them (model: $6\times$ tokens $\times$ matrix parameters, plus attention; statistics over the first micro-batch; eigendecompositions at $9n^3$; `scripts/analysis/flops.py`).

F^{-1} and Cayley cost $O(n^3)$ per matrix and step, against the model's $O(Nn^2)$ for N tokens, so their share grows with width at a fixed batch. With data parallelism the forward and backward pass is split across GPUs. The optimizer's matrices must then be split too (one GPU per matrix); if they are replicated on every GPU, as Pion's step is, their share grows with the number of GPUs. Statistics are the largest term in FLOPs but run on tensor cores in bf16; eigendecompositions are the smallest but latency-bound. This is why Tables 17 and 9 rank the parts differently.

[1] Seconds per step come from one job on one GPU (RTX PRO 6000), every configuration in turn for 600 steps of the 3 000-step schedule without evaluation, Pion first and last (`scripts/paper/timing.sbatch`).

Table 9. Optimizer FLOPs per step as a share of the model's forward and backward pass; the last row is the ratio of whole training steps. The squared Cayley retraction is counted at two Newton–Schulz iterations; the squarings of early large-angle steps are not counted.

	LLaMA-60M
Pion	1.2%
Natural Pion	10.5%
of which statistics / F^{-1} / Cayley / refresh	5.7% / 1.9% / 1.9% / 0.4%
of which generators and rotation (as Pion)	0.5%
whole training step, Natural Pion / Pion	1.09

5. Related work

Orthogonal equivalence training. POET (Qiu et al., 2025), following orthogonal fine-tuning (Qiu et al., 2023), reparameterises $W = RW_0P$ with orthogonal R and P, built from skew-symmetric parameters by a truncated Cayley–Neumann series; POET-X (Qiu et al., 2026) makes it cheaper in memory and compute. Pion (Shi et al., 2026) turns the same principle into an optimizer, with updates in the Lie algebra. ISO (Zhu et al., 2026) keeps the spectrum of a pretrained model fixed by writing $W = U\Sigma_0V^\top$, trains the frames with AdamW or Muon and retracts them by a polar map; on reinforcement learning from verifiable rewards with Qwen3-4B it reaches Pion's final accuracy faster. None of these uses a curvature metric: all move along the gradient with a first-order or orthogonalised preconditioner. Natural Pion keeps Pion's manifold and changes the metric to the Fisher.

Natural gradient on manifolds and Kronecker factorisations. The natural gradient (Amari, 1998) has a long line of approximations. K-FAC (Martens & Grosse, 2015) factorises a layer's Fisher into input and output-gradient moments; Shampoo (Gupta et al., 2018) builds Kronecker factors from the gradient itself. For orthogonality-constrained parameters, the natural Riemannian gradient goes back to ICA on Stiefel manifolds (Amari, 1999), and gradient flows that preserve eigenvalues or singular values are classical (Helmke & Moore, 1994, Ch. 2–3). Riemannian natural gradient methods (Hu et al., 2024) define the Fisher on a manifold's tangent space, use a retraction, and derive a K-FAC-style Kronecker approximation for Grassmann manifolds. Natural Pion is an instance of that framework on a different manifold, the orbit of a matrix under two-sided rotations. There the K-FAC Fisher on the generators is the two-term operator of Proposition 1, not a Kronecker product, and Section 3.2 inverts it exactly. We did

not find this operator, or a natural gradient for orthogonal-equivalence updates, in prior work; its inverse, by contrast, is the classical solution of a generalized Lyapunov equation (Simoncini, 2016).

Curvature-aware orthogonalised updates. A recent family puts Kronecker curvature into Muon's additive step. FISMO (Xu et al., 2026) fits Kronecker factors to gradient outer products and takes $W \leftarrow W - \eta P^{-1/2}\,\mathrm{Polar}(\tilde{G})\,Q^{-1/2}$ under a spectral-norm trust region in that metric. Go-MUON (Che, 2026) applies Muon between inverse fourth roots of K-FAC's activation and output-gradient moments. Mousse (Zhang et al., 2026) runs Muon in Shampoo-whitened coordinates. All three are additive and change the weight spectrum; Natural Pion's update is a rotation and preserves it.

Orthogonalised additive updates. Muon (Jordan et al., 2024) replaces the update by its orthogonal polar factor, $W \leftarrow W - \eta UV^\top$, which equalises the update's singular values but leaves the weight's spectrum free.

Grafting. Natural Pion is a graft: the direction of the natural gradient with Pion's step length. Grafting (Agarwal et al., 2020) takes the direction of one optimizer and the step size of another. Shampoo is run with it in practice (Anil et al., 2020; Shi et al., 2023), and Eschenhagen et al. (2025) trace why to the preconditioner's stale and mis-scaled eigenvalues: a good direction with a bad scale, which is what we find for F^{-1} (Section 3.3). Muon and the modular norm (Large et al., 2024; Bernstein & Newhouse, 2024) set the step in the spectral norm; a spectral rule in units of the rotation angle is one of the alternatives in Section 4.4.

Retractions on the orthogonal group. The Cayley transform is a standard retraction on the orthogonal and Stiefel manifolds (e.g. Wen & Yin, 2013; Li et al., 2020). Pion tested it and reports modest gains over its truncated exponential. At Natural Pion's larger angles, the squared Cayley retraction is what preserves the spectrum (Section 3.4).

6. Limitations

- **Scale.** One model size (60M) and one data set. The full-length comparison is one seed; Table 2 gives four seeds at 3 000 steps.
- **One code base.** Our Pion ends 0.016 above the Pion paper's training loss; we compare within our setting only.
- **Only Pion as a baseline.** We compare against Pion in its published settings, with its lr chosen per budget from $\{10^{-3}, 2\cdot10^{-3}\}$ and AdamW's lr chosen for it at 3 000 and 15 000 steps (Table 3 at full length). Muon is shown for reference only and AdamW not at all. We report validation loss only, not downstream tasks.
- **The peak scale depends on the horizon.** Like a learning rate, it is set per training length (Section 4.1).
- **Other choices were made at shorter horizons.** Lie momentum 0.9, T_{fac}, the statistics window and the floor were chosen at 3 000 steps and not re-tuned at full length. AdamW's lr was chosen at 3 000 and 15 000 steps; Table 3 checks two smaller values at full length.
- **The K-FAC assumption.** K-FAC's curvature along our step is far from the exact one (Section 3.3), which is why we keep only the direction of $F^{-1}G$.
- **Cost at scale.** The ratio of 1.09x Pion's FLOPs per step is a 60M figure. The overhead grows with width and, unless the matrices are split across GPUs, with the number of GPUs (Section 4.5). At the shapes of a 1.7B model, one refresh of all matrices takes 58 s on one B200, about 2.3 s per step at $T_{\mathrm{fac}} = 25$, likely comparable to the model's own step at that scale.

7. Conclusion

Pion fixes the spectrum for the whole run but leaves open which rotation to take, and we find the natural gradient a better choice than the raw gradient. On the rotation generators, K-FAC's Fisher is a two-term operator that one generalized eigendecomposition per side inverts exactly. Its direction, combined with Pion's step length and a retraction that preserves the spectrum at the larger angles, gives an optimizer that reaches Pion's losses on LLaMA-60M in 1.9–2.1x fewer steps at 1.09x its floating-point operations per step.

Future work. Reinforcement learning from verifiable rewards is the natural next test: a step costs minutes of generation, so saved steps (Section 4.3) matter most, and the natural policy gradient (Kakade, 2001) is its precedent. Larger models are the other: the optimizer's $O(n^3)$ work has to be split across devices (Section 6), and on an $m \times n$ matrix with $m > n$ the wide side can be solved in $O(m^2 n)$, since rotations outside W's column space do not move it.

Acknowledgements

Computations were performed on the Aphrodite HPC system of The Cyprus Institute (project p330).

References

Agarwal, N., Anil, R., Hazan, E., Koren, T., and Zhang, C. Disentangling adaptive gradient methods from learning rates. *arXiv:2002.11803*, 2020.

Amari, S. Natural gradient works efficiently in learning. *Neural Computation*, 10(2):251–276, 1998.

Amari, S. Natural gradient learning for over- and under-

complete bases in ICA. *Neural Computation*, 11(8): 1875–1883, 1999.

Anil, R., Gupta, V., Koren, T., Regan, K., and Singer, Y. Scalable second order optimization for deep learning. *arXiv:2002.09018*, 2020.

Bernstein, J. and Newhouse, L. Old optimizer, new norm: An anthology. In *OPT Workshop*, 2024. arXiv:2409.20325.

Che, T. Second-order Muon done right: A principled marriage of spectral geometry and curvature. *arXiv:2608.09763*, 2026.

Eschenhagen, R., Defazio, A., Lee, T.-H., Turner, R. E., and Shi, H.-J. M. Purifying Shampoo: Investigating Shampoo's heuristics by decomposing its preconditioner. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.03595.

Golub, G. H. and Van Loan, C. F. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.

Gupta, V., Koren, T., and Singer, Y. Shampoo: Preconditioned stochastic tensor optimization. In *International Conference on Machine Learning (ICML)*, 2018.

Helmke, U. and Moore, J. B. *Optimization and Dynamical Systems*. Springer, 1994.

Higham, N. J. The scaling and squaring method for the matrix exponential revisited. *SIAM Journal on Matrix Analysis and Applications*, 26(4):1179–1193, 2005.

Hu, J., Ao, R., So, A. M.-C., Yang, M., and Wen, Z. Riemannian natural gradient methods. *SIAM Journal on Scientific Computing*, 46(1):A204–A231, 2024. arXiv:2207.07287.

Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., and Bernstein, J. Muon: An optimizer for hidden layers in neural networks. https://kellerjordan.github.io/posts/muon/, 2024.

Kakade, S. A natural policy gradient. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2001.

Large, T., Liu, Y., Huh, M., Bahng, H., Isola, P., and Bernstein, J. Scalable optimization in the modular norm. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.14813.

Li, J., Li, F., and Todorovic, S. Efficient Riemannian optimization on the Stiefel manifold via the Cayley transform. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:2002.01113.

Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.

Martens, J. and Grosse, R. Optimizing neural networks with Kronecker-factored approximate curvature. In *International Conference on Machine Learning (ICML)*, 2015. arXiv:1503.05671.

Qiu, Z., Liu, W., Feng, H., Xue, Y., Feng, Y., Liu, Z., Zhang, D., Weller, A., and Schölkopf, B. Controlling text-to-image diffusion by orthogonal finetuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.07280.

Qiu, Z., Buchholz, S., Xiao, T. Z., Dax, M., Schölkopf, B., and Liu, W. Reparameterized LLM training via orthogonal equivalence transformation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.08001.

Qiu, Z., Liu, L., Weller, A., Shi, H., and Liu, W. POET-X: Memory-efficient LLM training by scaling orthogonal transformation. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2603.05500.

Raffel, C. et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 2020.

Shazeer, N. GLU variants improve transformer. *arXiv:2002.05202*, 2020.

Shi, H.-J. M., Lee, T.-H., et al. A distributed data-parallel PyTorch implementation of the distributed Shampoo optimizer for training neural networks at-scale. *arXiv:2309.06497*, 2023.

Shi, K., Li, H., Qiu, Z., Wen, Y., Buchholz, S., and Liu, W. Pion: A spectrum-preserving optimizer via orthogonal equivalence transformation. *arXiv:2605.12492*, 2026.

Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv:1909.08053*, 2019.

Simoncini, V. Computational methods for linear matrix equations. *SIAM Review*, 58(3):377–441, 2016. doi: 10.1137/130912839.

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. arXiv:2104.09864.

Touvron, H., Lavril, T., Izacard, G., et al. LLaMA: Open and efficient foundation language models. *arXiv:2302.13971*, 2023.

Wen, Z. and Yin, W. A feasible method for optimization with orthogonality constraints. *Mathematical Programming*, 142:397–434, 2013.

Xu, C., Yan, W., and Zhang, Y.-J. A. FISMO: Fisher-structured momentum-orthogonalized optimizer. *arXiv:2601.21750*, 2026.

Zhang, B. and Sennrich, R. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1910.07467.

Zhang, Y., Xing, S., Huang, J., Lv, K., Zhou, Y., Qiu, X., Guo, Q., and Chen, K. Mousse: Rectifying the geometry of Muon with curvature-aware preconditioning. *arXiv:2603.09697*, 2026.

Zhu, H., Cong, W., Sha, Z., et al. ISO: An RLVR-native optimization stack. *arXiv:2607.19331*, 2026.

A. Derivation of Proposition 1

We derive the Fisher of the input-side generator $\Omega_{\text{in}} \in \mathfrak{so}(n)$ in five steps; the output side follows by symmetry.

Step 1: the Fisher as a covariance on $\mathfrak{so}(n)$. By Section 2.1, the gradient of token t's loss with respect to Ω_{in} is

$$g_t = \operatorname{skew}(W^\top \delta_t \, x_t^\top) \in \mathfrak{so}(n).$$

The (empirical) Fisher on $\mathfrak{so}(n)$ is the second moment of g_t, a linear operator on $\mathfrak{so}(n)$:

$$F_{\text{in}}(X) = \mathbb{E}_t\big[\langle g_t, X\rangle \, g_t\big], \qquad X \in \mathfrak{so}(n),$$

so $\langle X, F_{\text{in}}(X)\rangle = \mathbb{E}_t[\langle g_t, X\rangle^2]$ is the mean square of the directional derivative along X. In coordinates it is an $\frac{n(n-1)}{2} \times \frac{n(n-1)}{2}$ matrix, about $1.7 \cdot 10^{10}$ entries for $n = 512$.

Step 2: each token's generator is a bivector. Write $u_t = W^\top \delta_t \in \mathbb{R}^n$, the output gradient pulled back to the input space. Then

$$g_t = \tfrac{1}{2}\big(u_t x_t^\top - x_t u_t^\top\big) = \tfrac{1}{2}\, u_t \wedge x_t,$$

the antisymmetrised outer product of two vectors. Its pairing with a skew X is a single bilinear form: since $x^\top X u = -u^\top X x$,

$$\langle g_t, X\rangle = \tfrac{1}{2}\big(u_t^\top X x_t - x_t^\top X u_t\big) = u_t^\top X x_t.$$

Step 3: expand. Substitute both expressions into Step 1 and move the scalar $u_t^\top X x_t$ next to the vector it pairs with:

$$\begin{aligned} F_{\text{in}}(X) &= \tfrac{1}{2}\,\mathbb{E}_t\big[(u_t^\top X x_t)\, u_t x_t^\top\big] - \tfrac{1}{2}\,\mathbb{E}_t\big[(u_t^\top X x_t)\, x_t u_t^\top\big] \\ &= \tfrac{1}{2}\,\mathbb{E}_t\big[u_t u_t^\top \, X \, x_t x_t^\top\big] - \tfrac{1}{2}\,\mathbb{E}_t\big[x_t x_t^\top \, X^\top u_t u_t^\top\big]. \end{aligned}$$

Step 4: the K-FAC assumption. As in K-FAC, assume the layer's input x_t and its output gradient δ_t (hence u_t) are independent. Each expectation then factorises:

$$\begin{aligned} \mathbb{E}_t\big[uu^\top X x x^\top\big] &= \mathbb{E}[uu^\top]\, X\, \mathbb{E}[xx^\top] = SXA, \\ \mathbb{E}_t\big[xx^\top X^\top uu^\top\big] &= AX^\top S, \end{aligned}$$

with $A = \mathbb{E}[xx^\top]$ and $S = \mathbb{E}[uu^\top] = W^\top \mathbb{E}[\delta\delta^\top] W = W^\top D W$.

Step 5: simplify. With $X^\top = -X$ the two terms add:

$$F_{\text{in}}(X) = \tfrac{1}{2}\big(SXA - AX^\top S\big) = \tfrac{1}{2}\big(SXA + AXS\big).$$

The output side. For $\Omega_{\text{out}} \in \mathfrak{so}(m)$, token t's generator is $\operatorname{skew}(\delta_t x_t^\top W^\top) = \tfrac{1}{2}\,\delta_t \wedge v_t$, with $v_t = W x_t$ the input pushed forward to the output space. Steps 2–5 carry over with (δ, v) in place of (u, x), and $\mathbb{E}[vv^\top] = WAW^\top$.

- **Size.** The operator of Step 1 has $O(n^4)$ entries; Proposition 1 determines it by two $n \times n$ matrices.
- **Both moments enter both sides.** The output-gradient moment enters the input side through $S = W^\top D W$, and the input moment enters the output side through $V = WAW^\top$.
- **Numerical check of the algebra.** With x and δ drawn independently, so that the K-FAC assumption holds by construction, Proposition 1 matches a Monte Carlo estimate to relative error 0.7% (in) and 0.4% (out) at 4×10^5 samples. The pull-back identity holds to $4 \cdot 10^{-15}$ (`scripts/analysis/check_fisher.py`).

B. Setup and hyperparameters

Table 10. Experimental setup.

model	LLaMA-style (Touvron et al., 2023), 60M parameters: hidden 512, 8 layers, 8 heads, SwiGLU FFN 1376 (Shazeer, 2020), RoPE (Su et al., 2024), RMSNorm (Zhang & Sennrich, 2019), vocabulary 32 100 (T5 tokenizer)
data	C4 (Raffel et al., 2020), sequence length 256, batch 512 sequences (131 072 tokens per step) in two micro-batches of 256
budget	73 242 steps = 9.6B tokens: Pion's 60M budget
rotated	the 56 matrices inside the blocks (43.5% of parameters)
the rest	AdamW (Loshchilov & Hutter, 2019), lr $8 \cdot 10^{-3}$, chosen for Pion from $\{4 \cdot 10^{-3}, 8 \cdot 10^{-3}, 1.6 \cdot 10^{-2}\}$ at 3 000 and 15 000 steps (the Pion script drives AdamW with its single lr, 10^{-3}); betas (0.9, 0.95), weight decay 0.1, cosine to 1%; the same for every method
precision	bf16 model; Natural Pion: statistics from bf16 inputs with fp32 accumulation, F^{-1} products in bf16, other products before the rotation in TF32, the Cayley step and the rotation of W in fp32
metric	validation loss: token cross-entropy on held-out C4, in nats
hardware	one NVIDIA RTX PRO 6000 Blackwell per run

Table 11. Pion learning rate (validation loss, seed 0).

Pion steps	$5 \cdot 10^{-4}$	10^{-3}	$2 \cdot 10^{-3}$	$4 \cdot 10^{-3}$
3 840	3.8413	3.7804	3.7805	4.1464
19 200	–	3.4960	3.4862	3.8366

Table 12. Hyperparameters.

	Natural Pion	Pion
rotated-matrix step	RMS rule, peak scale θ_0 per horizon (Section 4.1), cosine to 1% of the peak	RMS rule, lr 10^{-3} or $2 \cdot 10^{-3}$ per budget (scale 0.2· lr), cosine to 1%
momentum	Lie, 0.9	Lie, 0.9
curvature	T_{fac} 25; A, D EMA 0.5 over the 6 steps before a refresh; floor 10^{-3}	–
retraction	squared Cayley (Section 3.4)	truncated exponential, degree 2
update	both sides every step	both sides every step
other parameters	AdamW $8 \cdot 10^{-3}$, (0.9, 0.95), wd 0.1, cosine to 1%	same
gradient clipping	1.0	1.0
warmup	none	none

C. Making it fast, step by step

This appendix goes through the changes of Section 4.5 in the order they were made, with what each saves and what it costs in loss. Unless stated otherwise, numbers are at 3 000 steps, seed 0, Lie momentum 0.9, on one RTX PRO 6000. Seconds per step are the mean time between logged steps of the training runs themselves, without the slowest and fastest tenth (evaluation pauses).

C.1. Where the time goes

T_{fac} is the number of steps between basis refreshes (C.2).

Table 13. Seconds per step by part, over 400 profiled steps with GPU-synchronised timers (the synchronisation inflates the parts slightly).

part	exact form ($T_{\text{fac}} = 1$)	$T_{\text{fac}} = 25$
basis refresh (generalized eigendecompositions)	0.581	0.025
step scale (RMS rule)	0.007	0.006
Cayley retraction	0.053	0.053
F^{-1}	0.015	0.014
generators, momentum, rotation (as in Pion)	0.011	0.011
forward and backward (with the hooks that accumulate A and D), AdamW, data	0.555	0.557
whole step (profiled run)	1.22	0.67

C.2. Amortised refresh

The bases (P, λ) are recomputed every T_{fac} steps instead of every step. In between, F^{-1} uses the last bases with the current W, G and momentum. This is the standard amortisation of K-FAC, which recomputes its inverse every 20 iterations (Martens & Grosse, 2015, Sec. 13).

Table 14. Refresh cadence (Lie momentum 0.9; the exact form at the best scale of its own grid, the others at the method's scale, $6.0 \cdot 10^{-4}$).

refresh	val	s / step
every step (exact form)	3.6860	1.22
every 10 steps	3.6782 (-0.008)	0.70
every 25 steps	3.6903 ($+0.004$)	0.67

C.3. Statistics only where they are read

Only a refresh reads A and D, so they should describe the network at the refresh, not its history: older statistics belong to weights that have since moved. A single step already gives a well-conditioned estimate. Each is a gram over the 65 536 tokens of a micro-batch, 48 times the largest dimension of a rotated matrix (1376), so a long average removes little noise and keeps stale statistics. We therefore give both an EMA of 0.5, about two batches of memory, and accumulate them only in the 6 steps before each refresh (6 of every 25 steps). An observation more than 6 steps old carries less than 2% of the weight, so a longer window would add almost nothing. Neither number is tuned, and we did not try shorter windows. K-FAC keeps a longer memory, an EMA with decay $\min(1 - 1/k, 0.95)$ at iteration k (Martens & Grosse, 2015, Sec. 5); Table 15 includes 0.95 (for A in the reference row, for D in the last), at the same loss.

Table 15. Statistics windows ($T_{\text{fac}} = 25$, RMS scale $6.0 \cdot 10^{-4}$).

change ($T_{\text{fac}} = 25$)	EMA of A / D	val	s / step
both every step (reference)	0.95 / 0.5	3.6903	0.668
D only in the last 6 steps	0.95 / 0.5	3.6853	0.607
A only in the last 6 steps	0.5 / 0.5	3.6874	0.630
A and D only in the last 6 steps	0.5 / 0.5	**3.6867**	**0.568**
A every second step	0.9025 / 0.5	3.6875	0.643
D with a longer EMA, both every step	0.95 / 0.9, 0.95	3.6925 / 3.6998	0.668 / 0.667

The largest change from the reference is 0.009, so we take the cheapest option. (A every second step uses 0.95^2 to keep its time constant.)

C.4. Batching by shape

The 56 rotated matrices come in a few shapes. F^{-1} and the Cayley solves run as one batched call per shape instead of one call per matrix. Only round-off changes: 3.6872 against 3.6867. On its own this does not shorten the training step (0.568 s without, 0.568 s with); it pays off together with the batched scale and σ of C.5 (Table 17).

C.5. The step's scale, batched

Pion's rule needs $\|WX_{\text{in}} + X_{\text{out}}W\|_F$ for every matrix. It is batched by shape, with c kept on the GPU (no host synchronisation per matrix). $\sigma = \max(\|X_{\text{in}}\|_2, \|X_{\text{out}}\|_2)$ does not set the step. It estimates the step's largest angle from below, which decides how the Cayley step is computed (Section 3.4); an underestimate leaves the last factor slightly above the 0.2 threshold, where two iterations are still accurate to 10^{-6} at 0.4. The exact spectral norm (`matrix_norm(X, 2)`) would take 98% of the optimizer step on these shapes; power iteration on $X^\top X$ needs only matrix-vector products and costs 197x less. Started cold, it converges badly on skew matrices, whose singular values come in equal pairs: on a random 512×512 skew matrix its relative error is 21.0% after one iteration, 6.1% after five and 0.5% after twenty. Warm-started from the previous step's vector, one iteration reaches $2 \cdot 10^{-5}$. Natural Pion runs 2 warm iterations, and 50 right after a refresh, when X jumps. Batched, the scale and σ cost 0.005 s per optimizer step (Table 17). The exact Cayley solve of the earlier steps does not need σ, so it is not computed there.

C.6. Cheaper statistics and lower precision before the rotation

In the 6 steps before a refresh, every rotated matrix forms two grams, A over its inputs and D over its output gradients, each over the 65 536 tokens of the step's first micro-batch. In true fp32 these run without tensor cores.

One A per input. The query, key and value projections read the same normalised input, and so do the gate and up projections. So q, k, v share one A, which was formed three times, and gate and up share another, formed twice. Natural Pion forms each once per input tensor; the trajectory is bit-identical (`tests/test_natural_pion.py`).

bf16 inputs. Rounding the *assembled* A to bf16 does not work: its entrywise error, about 2^{-8} of the largest entries, exceeds the floor $10^{-3}\lambda_{\max}$, so the small eigenvalues the floor keeps become noise (Appendix D). Rounding the *inputs* is different. The gram of bf16 vectors is still a gram, so A stays positive semidefinite, and the rounding adds a noise covariance of relative size about $(2^{-8})^2 \approx 1.5 \cdot 10^{-5}$, far below the floor. We round activations and output gradients to bf16 and multiply them with fp32 accumulation and an fp32 result.

Table 16. Statistics inputs (seed 0, one A per input).

statistics	val, 3 000 steps	val, 15 000 steps	s / step
fp32 inputs	3.6826	3.4490	0.521
bf16 inputs (Natural Pion)	3.6878 (+0.005)	3.4465	0.498

bf16 inputs save 0.023 s per step.

Lower precision before the rotation, fp32 for the rotation. Everything up to the Cayley step only sets the direction, so it may be approximate: the four products of F^{-1} run in bf16 (the result is made exactly skew in fp32), and the other products before the rotation in TF32. The Cayley step and the rotation of W run in true fp32, because in TF32 the products of $R_{\text{out}}WR_{\text{in}}$ alone move the spectrum. Compared with fp32 throughout, the validation loss changes by $+0.0102$ at 3 000 steps (one run each; the method's four seeds span 0.006 there) and by $+0.0005$ at 15 000.

C.7. The optimizer step alone

Table 17. The optimizer step alone, by part (s; synthetic statistics on the shapes of the model, GPU-synchronised timers; `scripts/probes/optimizer_step_bench.py`).

part	after C.4	Natural Pion
statistics A, D	0.060	0.018
basis refresh (per step)	0.023	0.023
Cayley	0.053	0.025
F^{-1}	0.015	0.009
step scale (and σ)	0.007	0.005
generators, momentum, rotation (as in Pion)	0.011	0.009
total	0.168	**0.089**
Pion, whole optimizer step	–	0.029

Part of the optimizer's time overlaps the next step's host-side work, so a training step's time (0.498 against 0.473 s for Pion) is not the sum of this table.

D. Implementation details

- **Statistics.** A forward pre-hook on each rotated linear layer feeds A, and a hook on its output gradient feeds D, from one micro-batch per step. Both accumulate in fp32 with autocast disabled. A bf16 product that returns a bf16 matrix has an entrywise error of about 2^{-8} of the largest entries, above the floor; in training it even made eigenvalues of A negative (step 499: thousands over the 56 matrices). Rounding only the inputs (Appendix C.6) does neither. Scaling D by N^2 undoes the $1/N$ that autograd puts on a mean loss; with the loss also divided over our two micro-batches, D is a further factor 4 small. The RMS rule, the angle rules, the floor and the damping are unaffected by a common scale of D. Among the other length rules of Table 6, it rescales θ by 2 for the Fisher norm and by 4 for the fixed and quadratic-model rules, each of which was tuned over its own grid of θ.
- **Rounding bias of the retraction.** Over random rotations (1376×1376, angle 0.05), $\mathrm{tr}(R^\top R - I)/n$ averages $+1.7 \cdot 10^{-7}$ for the exact solve (171 standard errors from zero), $+1.4 \cdot 10^{-6}$ for standard Newton–Schulz, and $+6.3 \cdot 10^{-11}$ (0.7 standard errors) in the $\Phi = Z - I$ form of Section 3.4 (`scripts/probes/retraction_bias.py`).
- **Code.** `ngd_pion/natural_pion.py`: `NaturalPionFast` is Algorithm 1, the method of every experiment; the exact form is the same class with the basis rebuilt and the statistics collected at every step, and the exact Cayley solve (`t_fac` 1, `stats_window` null, `retraction` cayley; rows `c-alg1-*`). `NaturalPion` is the earlier reference with the angle rule $c = \theta/\sigma$, kept for the tests; every job and run record is listed in `scripts/paper/README.md` and `runs-record/`. Every measured number in this paper is computed from the run records by `scripts/analysis/paper_numbers.py` (the list, with sources, is `docs/paper/numbers.md`).